

Outline Import and Normalization

This section explains how the machine accepts legacy outlines and turns them into the canonical work object used everywhere else in the system. The importer is intentionally conservative: it does not try to preserve every surface detail of a source format. Instead, it extracts structure, intent, and content, then normalizes those elements into the same schema that conversational intake produces.

The supported inputs reflect the kinds of material authors already have on hand. A project may begin as legacy YAML, a Markdown document with heading structure, a numbered outline, or even a block of raw text. Each of these forms carries useful information, but each encodes that information differently. The importer therefore performs two related tasks: it parses the source format, and it maps the parsed result into canonical work YAML.

For legacy YAML, the importer reads the existing keys and compares them against the current work schema. Fields that already match the canonical model can be carried forward directly. Fields that are older, renamed, or nested differently are translated into the current structure. This makes migration possible without forcing authors to rewrite a project by hand before the system can use it. When the source file is close to the current schema, the importer behaves like a translator; when it is older or partially incompatible, it behaves like a migration layer.

Markdown imports are treated as structural documents. Heading levels become outline nodes, paragraph text becomes section content, and list structure is preserved when it contributes to meaning. The importer uses the heading hierarchy to infer parent-child relationships, so a document organized with #, ##, and deeper headings can be converted into a recursive outline tree. The goal is not to reproduce Markdown as Markdown, but to recover the authorial structure that the headings imply. In practice, this means the importer treats headings as signals of hierarchy rather than as formatting to be preserved verbatim.

Numbered outlines are normalized in a similar way. A sequence such as 1., 1.1, and 1.1.1 already expresses hierarchy, but the numbering itself is not the canonical representation. The importer uses the numbering to determine nesting, then stores the result as structured nodes with stable identifiers and content fields. Once normalized, the outline no longer depends on the visual numbering scheme that produced it. This is important because the canonical work object must remain stable even if the author later rennumbers or reorders the source outline.

Raw text is the least structured input, so the importer applies the most cautious treatment. It can preserve the text as content, but it cannot reliably infer deep hierarchy unless the source includes explicit cues such as blank-line separation, repeated labels, or numbering. In practice, raw text imports are often used as a starting point for manual refinement: the importer captures what it can, then leaves the resulting work object ready for later editing. That restraint is deliberate, because false structure is more damaging than missing structure.

Across all input types, normalization serves the same purpose: it produces a single canonical work YAML representation rooted at **work**. That representation can then be validated, diffed, routed through agents, and compiled without special cases for the original source format. The importer therefore acts as a boundary between heterogeneous author input and the machine's shared internal model.

This normalization step is also where migration reports become useful. When the importer encounters missing fields, ambiguous hierarchy, or source elements that do not map cleanly into the canonical schema, it can record those issues for review. The result is a work object that is usable immediately, plus a record of what was inferred, translated, or left unresolved. In that sense, import is not merely conversion; it is the first disciplined act of editorial reconciliation in the system.

A simple example makes the distinction concrete. A legacy outline might arrive as a Markdown file with headings and paragraphs, or as YAML with nested sections already partially described.

The importer does not ask the author to choose a new format first. It reads the source as it is, identifies the structural cues available in that source, and emits the same canonical shape in both cases. That shared shape is what allows later stages of the system to work uniformly.

By the end of this process, the project is no longer a legacy file, a loose outline, or an unstructured note. It is a canonical work object that the rest of the Codynamic Book Machine can understand consistently. From that point forward, the book can move through validation, decomposition, agent work, and compilation without needing to remember how it began.

A useful way to think about the importer is as a normalization funnel. At the top of the funnel, the system accepts several familiar authoring forms; at the bottom, it emits one machine-readable work structure. The narrower output is not a loss of meaning so much as a reduction of ambiguity. The importer keeps the distinctions that matter for later processing, such as section boundaries, nesting, and content text, while discarding source-specific presentation details that would otherwise complicate downstream tooling.

That design also keeps the rest of the pipeline simpler. Once import has produced canonical work YAML, later components do not need separate code paths for Markdown, YAML, numbered notes, or freeform text. They can operate on one representation and rely on the importer to have already resolved the differences among source formats. In a system built around durable coordination, that early standardization is what makes the rest of the workflow predictable.